

Программирование и прототипирование ИТ-решений

Базовые типы данных, операторы, модель памяти, динамическая типизация

ЛЕКЦИЯ 2

Ахметов Ильнур Вазирович

План лекции

1 Зачем аналитику типы данных

2 Базовые типы: `int`, `float`, `str`, `bool`, `None` + практика

3 Операторы: арифметика, сравнение, логика

4 Явное и неявное преобразование типов

5 Модель памяти: переменная — это ссылка

6 Изменяемые и неизменяемые объекты, `id()` и `is`

7 Динамическая типизация

8 Сквозной кейс и типичные ошибки

Зачем аналитику типы данных

Тип данных определяет, какие значения может принимать переменная и какие операции над ними допустимы.

01

Что можно делать

От типа зависит поведение операторов: «+» для чисел — сумма, для строк — склейка.

02

Где и сколько памяти

Разные типы занимают разное место: число и строка хранятся в памяти по-разному.

03

Где чаще всего ошибаются

Ошибки типов — источник большинства багов: сложить число со строкой нельзя.

04

Как быстрее учиться

Понимание типов = чтение документации и предсказание поведения кода.

Пять базовых типов Python

int

Целые числа

Счётчики, номера, индексы. 42, -7, 2026

float

Дробные числа

Физические величины. 3.14, 142.5

str

Строки (текст)

Названия, коды, описания. "скважина"

bool

Логический тип

Флаги состояния «да/нет». True / False

None

«Ничего»

Отсутствие значения. None

int — целые числа

- ▶ Произвольная точность: в Python нет переполнения, как в C или Java — $10^{**}100$ считается без ошибок.
- ▶ Классика для счётчиков, количества объектов, номеров скважин, индексов.
- ▶ Деление нацело ($//$) и остаток (%) незаменимы, когда нужно «по N штук в группе».
- ▶ Тип определяет результат: $7 / 2$ — это float 3.5, а $7 // 2$ — int 3.

консоль Python

большие числа без переполнения

```
>>> big = 10 ** 100
>>> big + 1
1000...001 (101 цифра)
```

целочисленное деление и остаток

```
>>> 17 // 5      # 3
>>> 17 % 5       # 2
>>> divmod(17, 5) # (3, 2)
```

Распределение скважин по кустам

Задача: на месторождении 47 скважин, на каждый куст помещается по 5. Сколько полных кустов и сколько останется?

// $47 // 5 = 9$ — число полностью заполненных кустов.

% $47 \% 5 = 2$ — сколько скважин не вошли в полные кусты.

чётность $x \% 2 == 0$ — проверка «чётное ли число» (полезно для парных скважин).

точно int-арифметика точна: никаких погрешностей округления, в отличие от float.

решение

```
скважин = 47
```

```
в_кусте = 5
```

```
полных = 47 // 5    # 9
```

```
остаток = 47 % 5    # 2
```

```
чётное = 47 % 2 == 0 # False
```

```
итого_кустов = полных + (1 if остаток else 0)
```

```
# 10 (9 полных + 1 неполный)
```

float — числа с плавающей точкой

- ▶ Хранятся по стандарту IEEE 754 — как мантисса и порядок.
- ▶ Дебит, давление, температура, обводнённость — почти вся наша телеметрия.
- ▶ ⚠ Точность: 0.1 нельзя представить точно — накапливается погрешность.
- ▶ Для денег и точных расчётов — тип Decimal из модуля decimal.

консоль Python

знаменитая ловушка

```
>>> 0.1 + 0.2
0.30000000000000004
```

сравнивайте с допуском!

```
>>> round(0.1 + 0.2, 2) # 0.3
>>> abs(0.3 - (0.1+0.2)) < 1e-9 # True
```

точные деньги — Decimal

```
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2") # 0.3
```

Расчёт дебита: формула Дюпюи

Задача: оценить дебит скважины по формуле притока. Все физические величины — это float.

float проницаемость k , депрессия ΔP , вязкость μ — дробные физические величины.

math логарифм $\ln()$ из модуля `math` — стандартная математика.

формат `f"...{q:.3f}..."` — вывод с тремя знаками после запятой.

точность результат — приближённый float: для точных проектных расчётов — `Decimal`.

решение

```
import math
k = 150e-15      # проницаемость, м²
h = 12           # толщина пласта, м
dP = 3.5e6       # депрессия, Па
mu, Rk, rc = 0.0012, 250, 0.1

q = (2*math.pi*k*h*dP)
q = q / (mu*math.log(Rk/rc))
print(f"Дебит: {q:.6f} м³/с")
```

str — строки

- ▶ Неизменяемая последовательность символов: строку нельзя «дописать» — создаётся новая.
- ▶ Операции: «+» — склейка, «*» — повторение, len() — длина.
- ▶ Доступ по индексу и срезу: s[0], s[-1], s[1:4].
- ▶ Названия скважин, коды, описания — вся текстовая часть предметной области.

строковые операции

```
>>> well = "скважина"
>>> well.upper()
СКВАЖИНА
>>> well[0]          # с
>>> well[-1]         # а
>>> "№" + "12"
№12
>>> "па" * 4
папапапа
>>> f"Дебит: {142.5} т/сут"
Дебит: 142.5 т/сут
```

Разбор кода скважины

Задача: код скважины «K-12-НК» (куст — номер — назначение). Извлечь номер и собрать читаемое имя.

split() разбить строку по разделителю "-" → список частей.

индексы parts[0] — куст, parts[1] — номер, parts[2] — назначение.

int() превратить строку «12» в число 12 для расчётов.

f-строка собрать читаемое описание из кусочков.

решение

```
code = "K-12-НК"
parts = code.split("-")
# ["K", "12", "НК"]

nom = int(parts[1])    # 12
kust = parts[0]        # "K"

print(f"Скважина №{nom} куста {kust}")
# Скважина №12 куста K

print(code.startswith("K")) # True
```

bool — логический тип

- ▶ Всего два значения: True и False — результат любого сравнения.
- ▶ «Скважина в работе?», «данные валидны?» — это булевы флаги.
- ▶ Правило truthy/falsy: 0, "", None, [] считаются False — остальное True.
- ▶ Используется в условиях if / while — о них в следующей лекции.

консоль Python

```
>>> 5 > 3
True
>>> 5 == 5.0
True    # равны по значению
>>> bool("скважина")
True    # непустая строка
>>> bool("")
False   # пустая строка
>>> bool(0), bool(None)
(False, False)
```

Валидация: давление в норме?

Задача: проверить, что забойное давление лежит в допустимом диапазоне и режим добычи корректен.

цепочка `min_p <= p_zab <= max_p` — сравнение «в диапазоне» одной строкой.

сравнение `p_plast > p_zab` — пластовое давление выше забойного → добыча идёт.

and объединение условий: оба должны быть истинны.

флаг результат — bool, который потом пойдёт в if / журналирование.

решение

```
p_plast = 25.0    # МПа
p_zab   = 18.5    # МПа
min_p, max_p = 5.0, 30.0

в_диапазоне = min_p <= p_zab <= max_p
# True

депрессия   = p_plast > p_zab
# True

ok = в_диапазоне and депрессия
# True — режим в норме
```

None — «значения нет»

None — единственное значение типа NoneType. Оно означает «значение отсутствует» — это не ноль и не пустая строка.

1

Когда применять

Функция ничего не вернула; параметр без значения по умолчанию; «данные ещё не получены».

2

Как проверять

Только через is: `x is None`. Не используйте `==` — это отдельная история.

3

Частая ошибка

Путать None с 0 или `""` — это разные вещи. None — про «ничего», а не про «пусто».

пример использования

```
def find_well(name):
    # если не нашли — возвращаем None
    return None
```

```
if find_well("№12") is None:
    print("Скважина не найдена")
```

Арифметические операторы

+ сложение $7 + 3 = 10$

- вычитание $7 - 3 = 4$

***** умножение $7 * 3 = 21$

/ деление $7 / 2 = 3.5$ (всегда float)

// деление нацело $7 // 2 = 3$

% остаток $7 \% 2 = 1$

****** степень $2 ** 10 = 1024$

порядок + приоритет

```
>>> 7 + 3 * 2      # 13
>>> (7 + 3) * 2    # 20
>>> 7 / 2           # 3.5
>>> 7 // 2          # 3
>>> 7 % 2           # 1
>>> 2 ** 10         # 1024

>>> 2 ** 3 ** 2     # 512
>>> -2 ** 2         # -4
```

Сравнение и логические операторы

== != равно / не равно

< > меньше / больше

<= >= меньше или равно / больше или равно

and И — истина, если оба истинны

or ИЛИ — истина, если хотя бы одно

not НЕ — инверсия

приоритет not > and > or

консоль Python

```
>>> p, z = 250, 120 # давления
```

```
>>> p > z           # True
```

```
>>> p == z          # False
```

```
>>> p > 200 and z < 200
```

```
True # оба условия
```

```
>>> not (p > z)
```

```
False
```

```
>>> 0 or 42
```

```
42 # вернёт truthy-значение
```

Приоритет и скобки

1 (высший)

****** — возведение в степень

2

***** / **//** **%** — умножение и деление

3

+ **-** — сложение и вычитание

4

< **>** **<=** **>=** **==** **!=** — сравнения

5

not

6

and

7 (низший)

or

Правило

Чтобы не запоминать таблицу — используйте скобки: они делают порядок намеренным и читаемым.

a + b * c **≠** **(a + b) * c**

Явные скобки — признак зрелого кода.

Типичные путаницы:

-2 ** 2 = -4

2 ** 3 ** 2 = 512

Преобразование типов

int() "42" → 42

float() "3.14" → 3.14

str() 42 → "42"

bool() любое значение → True / False

⚠ int(3.99) = 3 — отброс дробной части, а не округление

⚠ int("42.5") упадёт с ошибкой (внутри нужен int)

явное преобразование

```
>>> int("42")      # 42
>>> float("3.14")  # 3.14
>>> str(42)        # "42"
>>> bool(0)        # False
>>> bool("x")       # True
```

```
>>> int(3.99)      # 3 (!)
>>> round(3.99)    # 4
```

```
>>> input() на входе — всегда str
>>> n = int(input("Число: "))
```

Ввод пользователя — всегда строка

Задача: посчитать годовой объём добычи из суточного дебита, введённого с клавиатуры.

input() всегда возвращает str — даже если пользователь ввёл число.

float() явное преобразование введённой строки в число для расчётов.

расчёт годовой объём = суточный дебит × 365.

ошибка пустой или нечисловой ввод → ValueError, нужна обработка (позже).

решение

```
raw = input("Дебит, т/сут: ")
# raw — это строка, например "142.5"

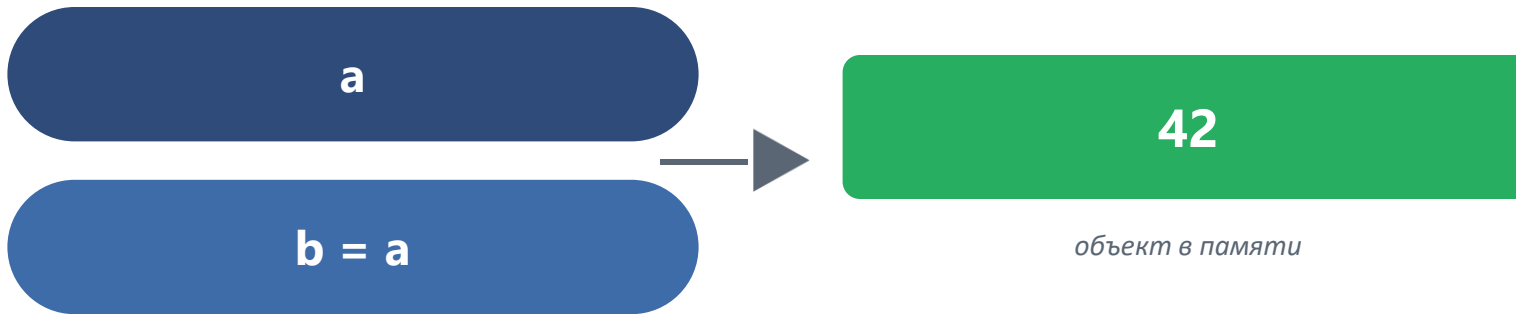
q = float(raw)      # 142.5 (число)
год = q * 365

print(f"За год: {год:.0f} т")
# За год: 52012 т

# без float() было бы:
# "142.5" * 365 → строка!
```

Переменная — это ярлык, а не коробка

Переменная не хранит значение — она лишь ссылка на объект в памяти. Одному объекту может соответствовать несколько имён.



демонстрация

```
a = 42
b = a      # b — второй ярлык на тот же объект
b is a     # True

a = 43     # a теперь указывает на новый объект
b          # 42 — объект b не изменился
```

Изменяемые и неизменяемые объекты

НЕИЗМЕНЯЕМЫЕ (immutable)

`int` • `float` • `str` • `bool` • `tuple`

Операция создаёт НОВЫЙ объект, а старый остаётся нетронутым.

демонстрация

```
s = "газ"
s.upper() # новый объект
s         # "газ" — не изменилась
```

ИЗМЕНЯЕМЫЕ (mutable)

`list` • `dict` • `set` (дальше в курсе)

Можно менять «на месте»: все ярлыки на объект видят изменение.

демонстрация

```
lst = [1, 2]
lst.append(3)
lst # [1, 2, 3]
```

Почему важно: неизменяемые типы безопасны при передаче в функции, изменяемые — требуют внимания.

id(), is и ==

id(x) числовой «адрес» объекта в памяти (в CPython — адрес)

is сравнивает ИДЕНТИЧНОСТЬ — тот же самый объект

== сравнивает ЗНАЧЕНИЕ — одинаковое содержимое

⚠ **is** ≠ **==** — частая причина трудноуловимых багов

Кстати, маленькие числа и короткие строки Python кэширует (интернирует) — поэтому

256 is 256 → True

равенство vs идентичность

```
>>> a = [1, 2, 3]
>>> b = a
>>> c = a.copy()

>>> a == b, a is b
(True, True)
>>> a == c, a is c
(True, False) # разные объекты!

>>> id(a), id(c)
14019..., 14028...
```

Динамическая типизация

Тип переменной определяется в момент выполнения по значению. Одна и та же переменная может последовательно хранить значения разных типов.

- ▶ Никаких объявлений: не пишем `int x = 42`, просто `x = 42`.
- ▶ `type(x)` — узнать тип, `isinstance(x, int)` — проверить принадлежность.
- ▶ Плюс: меньше кода, быстрее прототип, выше гибкость.
- ▶ Минус: ошибки типов всплывают только при выполнении — нужны тесты.

тип меняется на лету

```
x = 42
type(x)      # <class 'int'>
x = "скважина"
type(x)      # <class 'str'>
x = 3.14
type(x)      # <class 'float'>

isinstance(x, float) # True
```

Динамическая vs статическая типизация

PYTHON — ДИНАМИЧЕСКАЯ

```
x = 42          # тип выведен сам
```

```
x = "текст"     # можно сменить тип
```

- ✓ ошибки типов — в рантайме
- ✓ быстро писать, легко прототипировать
- ✓ нужны тесты для надёжности

JAVA / C# — СТАТИЧЕСКАЯ

```
int x = 42;     // тип объявлен
```

```
x = "текст";    // ошибка компиляции
```

- ✓ ошибки типов — на этапе сборки
- ✓ больше кода, но раньше ловим баги
- ✓ жёсткая типизация для больших систем

Оба подхода имеют право на жизнь. Python выбирает скорость разработки — именно поэтому он идеален для аналитики и прототипов.

Калькулятор дебита: все типы вместе

Соберём всё в один сценарий: строка-код, целое количество дней, дробный дебит и логический флаг состояния.

str well = "K-12-НК" — идентификатор скважины.

int days = 30 — количество дней в отчётном периоде.

float q = 142.5 — суточный дебит (дробное число).

bool active = True — флаг «в работе».

f-строка вывод: код, объём за месяц и состояние одной строкой.

мини-программа

```
well = "K-12-НК"
days = 30          # int
q = 142.5           # float
active = True       # bool

месяц = q * days
состояние = "в работе" if active else "стоп"

print(f"{well}: {месяц:.0f} т/мес, {состояние}")
# K-12-НК: 4275 т/мес, в работе
```

Предскажите результат

`"2" + "3"`

→ `"23"` – склейка строк

`2 + 3 * 4`

→ `14` – сначала умножение

`int(4.9)`

→ `4` – отброс дробной части

`0.1 + 0.2 == 0.3`

→ `False` – погрешность `float`

`7 / 2`

→ `3.5` – деление даёт `float`

`"a" * 3`

→ `"aaa"` – повторение строки

Дайте залу 10–15 секунд на каждый пример, затем покажите ответ. Это проверка понимания вживую.

Найдите и исправьте ошибку

✗ `age = input(); print(age + 1)`

✓ `age = int(input()); print(age + 1)`

✗ `s = "газ"; s.upper(); print(s)`

✓ `s = s.upper(); print(s) # "ГАЗ"`

✗ `print(5 / 2) # хочу целое`

✓ `print(5 // 2) # 2`

✗ `print("42" + 1) # TypeError`

✓ `print(int("42") + 1) # 43`

Три источника ошибок: (1) тип не тот (2) забыли присвоить результат (3) не тот оператор. Держите этот список в голове.

Типичные ошибки и как их избежать

TypeError

"42" + 1 → нельзя сложить str и int. Лечение: `int("42") + 1`.

Точность

$0.1 + 0.2 \neq 0.3$. Лечение: `round()` или `Decimal`.

is vs ==

`is` сравнивает объекты, `==` — значения. С числами может «случайно» работать.

int(3.99)

Это 3, а не 4 — отбрасывает дробную часть (не округляет).

Неизменяемость

`s.upper()` не меняет `s` — вернёт новую строку, которую надо присвоить.

Mutable-аргумент

Передали список в функцию и случайно изменили оригинал. Разберём с коллекциями.

Итоги лекции

01

Тип данных = правила поведения

int, float, str, bool, None — что допустимо и какие операции доступны.

02

Операторы возвращают результаты

Арифметика, сравнение, логика — и у них есть приоритет (помните про скобки).

03

Преобразование — осознанно

int(), float(), str(), bool() — явно, а не «само собой».

04

Переменная — ссылка, а не коробка

Один объект — много ярлыков. id(), is и == проясняют картину.

05

Изменяемость имеет значение

Строки неизменяемы; списки меняются на месте — помним об этом.